Toegepaste Informatica

UCLL HOGESCHOOL

Full-Stack Software Development

OpenAPI & Swagger: Streamlining API Documentation & Testing

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

CONTENTS

- The problem: The API Communication Gap
- OpenAPI & Swagger
- Implementation

Problem statement:
The API communications gap

THE API COMMUNICATIONS GAP

Problem 1: Developer Disconnect

Frontend and backend teams struggle to stay in sync. "What's the endpoint?" "What's the correct JSON structure?" "Did this parameter change?“

THE API COMMUNICATIONS GAP

Problem 2: Maintaining manual documentation

- Writing and maintaining static documentation (like a Wiki or Word doc) is tedious, error-prone, and quickly becomes outdated as the code evolves.
- There’s always a time gap between code deployment and updating the manual documentation.

OpenAPI & Swagger

OPENAPI

OpenAPI:

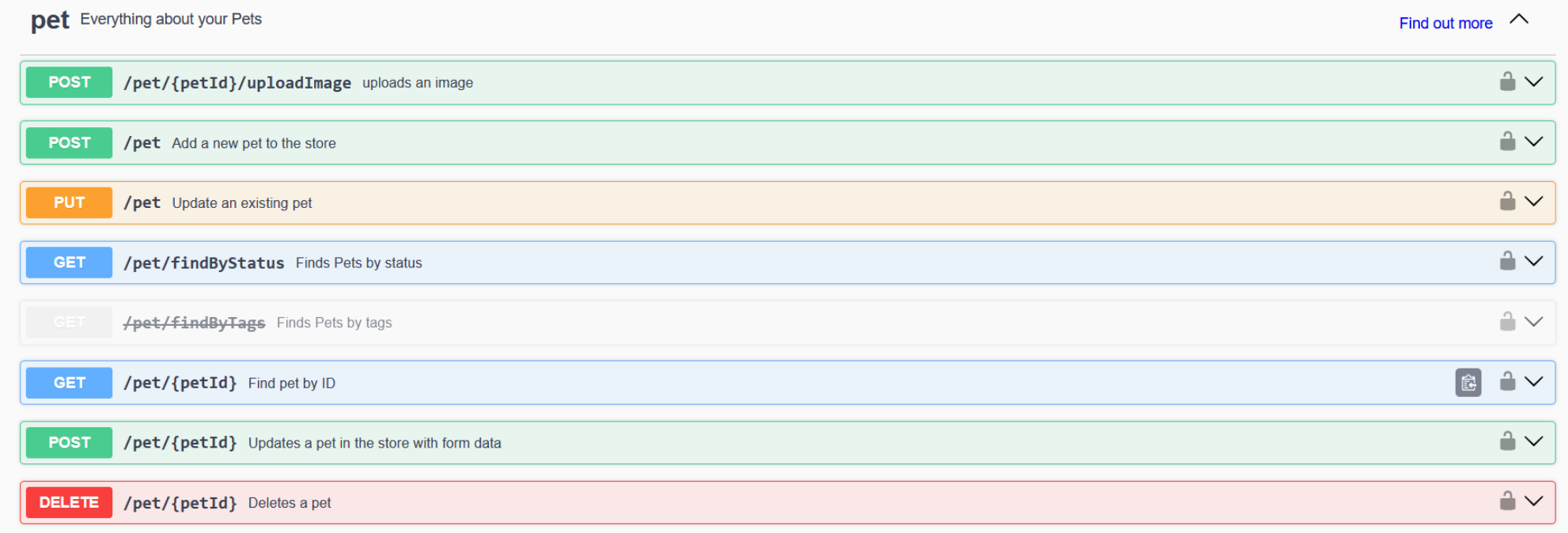
- A specification (like a blueprint) for building and documenting RESTful APIs.
- It defines endpoints, parameters, responses, and data models in a language-agnostic way.
- For reference: https://swagger.io/specification/

SWAGGER

- A set of tools built around the OpenAPI specification.
- The most famous tool is **Swagger UI**, which generates interactive, web-based documentation for your API.
- For reference: https://swagger.io/docs/

SWAGGER DEMO

Go to https://petstore.swagger.io/ to see swagger ui in action



ADVANTAGES

1-Click

Interactive Testing

Swagger UI provides an "Try it out" button for every endpoint. This allows anyone (devs, QAs, even product managers) to test API endpoints directly in the browser *without* writing any code or using external tools like Postman.

Single Source of Truth

The documentation is generated *from your code*. It's always in sync with your API's actual implementation.

Client SDK Generation

The OpenAPI spec can be used to automatically generate client libraries in various languages (Java, Python, JS).

Clear Contracts

Provides a clear, machine-readable contract for frontend teams and external consumers to build against.

Implementing Swagger in our Spring Boot app

ADDING THE DEPENDENCY

Add this dependency to your pom:

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.4</version>
</dependency>
```

THE RESULT

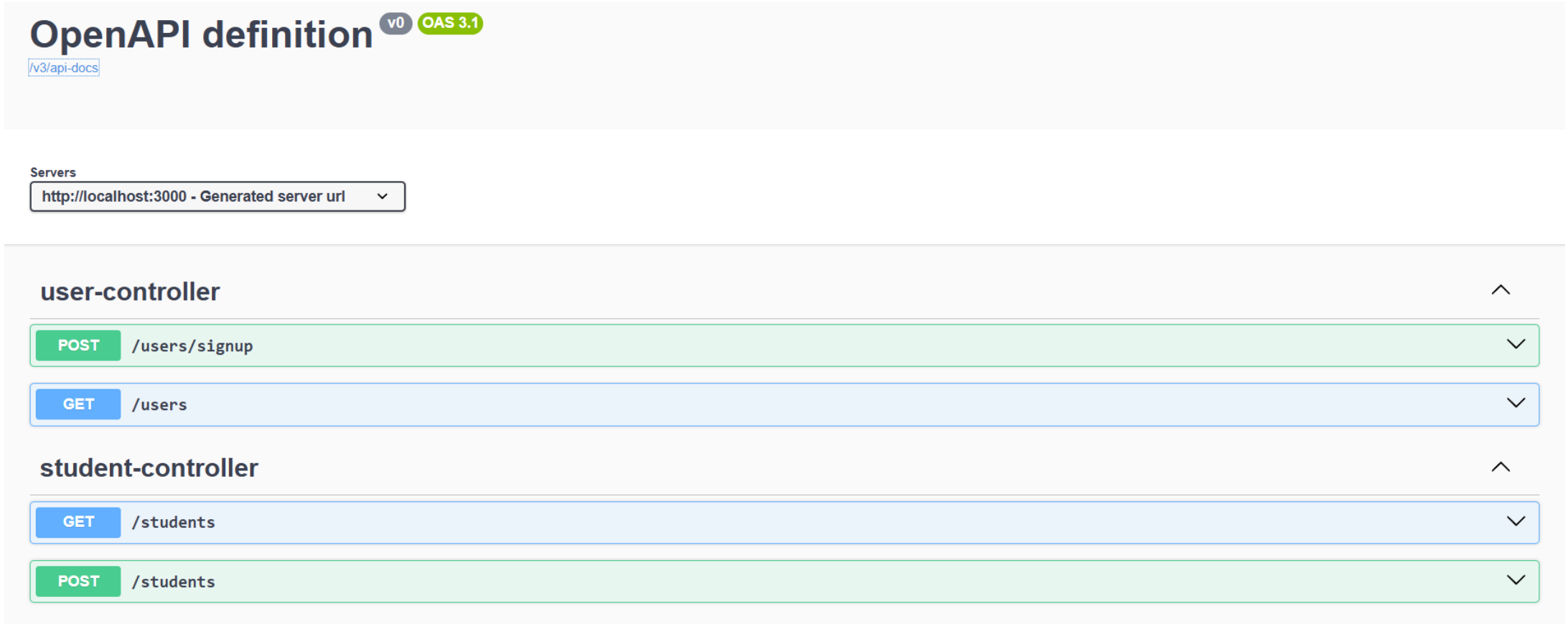
On /v3/api-docs: a json file describing you

Snippet:


```
"/students": {
  "get": {
    "tags": [
      "student-controller"
    ],
    "operationId": "getAllStudents",
    "responses": {
      "404": {
        "description": "Not Found",
        "content": {
          "*/*": {
            "schema": {
              "type": "object",
              "additionalProperties": {
                "type": "string"
              }
            }
          }
        }
      },
      "400": {
        "description": "Bad Request",
        "content": {
          "*/*": {
            "schema": {
              "type": "object",
              "additionalProperties": {
                "type": "string"
              }
            }
          }
        }
      },
      "200": {
        "description": "OK",
        "content": {
          "*/*": {
            "schema": {
              "type": "array",
              "items": {
                "$ref": "#/components/schemas/Student"
              }
            }
          }
        }
      }
    }
  }
},
```

THE RESULT

On /swagger-ui.html: The swagger ui page



ADDING TO THE AUTOGENERATED SWAGGER UI

To change the title, description, version, licence, ….

Add this to your project

```java
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("Courses API")
                        .version("1.0")
                        .description("API for managing courses, users, and schedules."));
    }
}
```

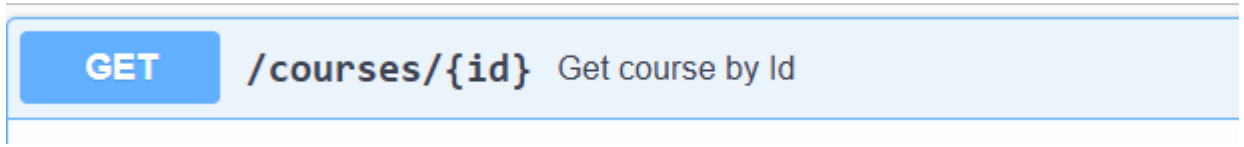
@Configuration marks a class as a source of bean definitions.

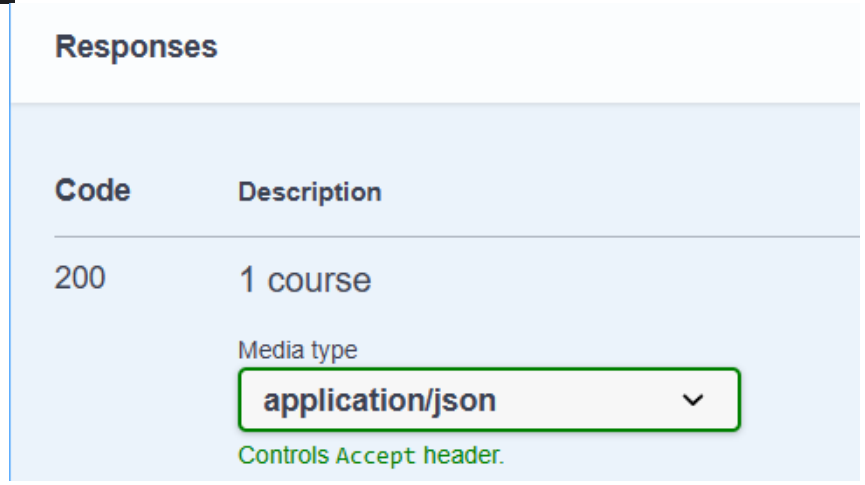
It tells Spring that this class contains methods annotated with @Bean, which produce beans to be managed by the Spring container.

ADDING DETAIL TO YOUR ENDPOINTS

`@Operation`	Describes what a single endpoint (method) does.
`@ApiResponse`	Describes a possible response (e.g., 200 OK, 404 Not Found).
`@Parameter`	Adds details to a method parameter (e.g., "User ID").

```java
@Operation(summary = "Get course by Id")
@ApiResponse(responseCode = "200", description = "List of Courses")
@GetMapping("/{id}")
public Course getCourseById(@PathVariable long id) {
    return courseService.getCourseById(id);
}
```





MANY ANNOTATIONS ARE AVAILABLE

```java
@Tag(name = "Books", description = "Book management APIs")
@RestController
@RequestMapping("/api/books")
public class BookController {
    @Operation(summary = "Get all books", tags = {"Books"})
    @ApiResponse(responseCode = "200", description = "Successful retrieval of books",
            content = @Content(mediaType = "application/json",
                    array = @ArraySchema(schema = @Schema(implementation = Book.class))))
    @GetMapping
    public List<Book> getBooks() {
        return List.of(
                new Book( id: 1,  title: "Spring Boot Essentials"),
                new Book( id: 2,  title: "Swagger in Action"));
    }

    @Operation(summary = "Add a new book")
    @ApiResponse(responseCode = "201", description = "Book added successfully")
    @PostMapping
    public String addBook(
            @RequestBody(description = "Book details", required = true) Book book) {
        return "Book added";
    }
}
```

Find more information about these tags, and others here:

https://masteringbackend.com/posts/spring-boot-swagger-a-complete-guide-to-api-documentation